

Distributed Systems - 27_10

A register is a shared value accessible by multiple processes with read and write permissions. So we want a memory location accessible by multiple processes. We want to implement this "primitive". The different processes will interact in the network. We'll implement this thing with messages, through a middleware known as Simulator.

The primitive Register will have two operations:

- Write(value t): write a certain value on the register
- Read: return the current value of the register

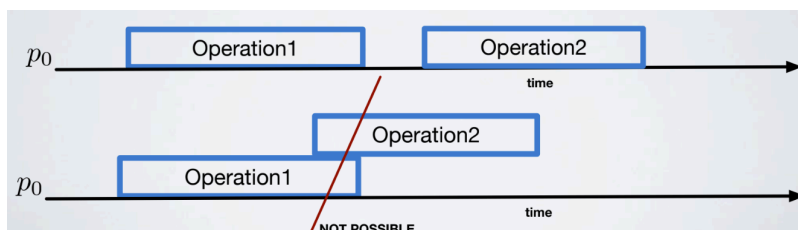
What does it mean "current"? It's not easy to define.

The operations we do on a value are not atomic in time. There will be a moment in which we start writing, and then for a certain amount of time we'll continue to write, until we have a local event "return" which states that an operation is complete. Operations are not blocking as they are now.

How does the read operation work? We start reading, giving no parameter (in contrary of write, where we give a parameter x we want to write on a certain value), and then after a certain time there is the return of a value v .

What are the assumptions?

- Initially a register is initialized to a certain value, like 0 or $\perp$. Also, it only stores positive numbers.
- When we write a value on the register, it's different from the ones we've seen before, so each written value is univocally identified.
- Operations on the same process are sequential.

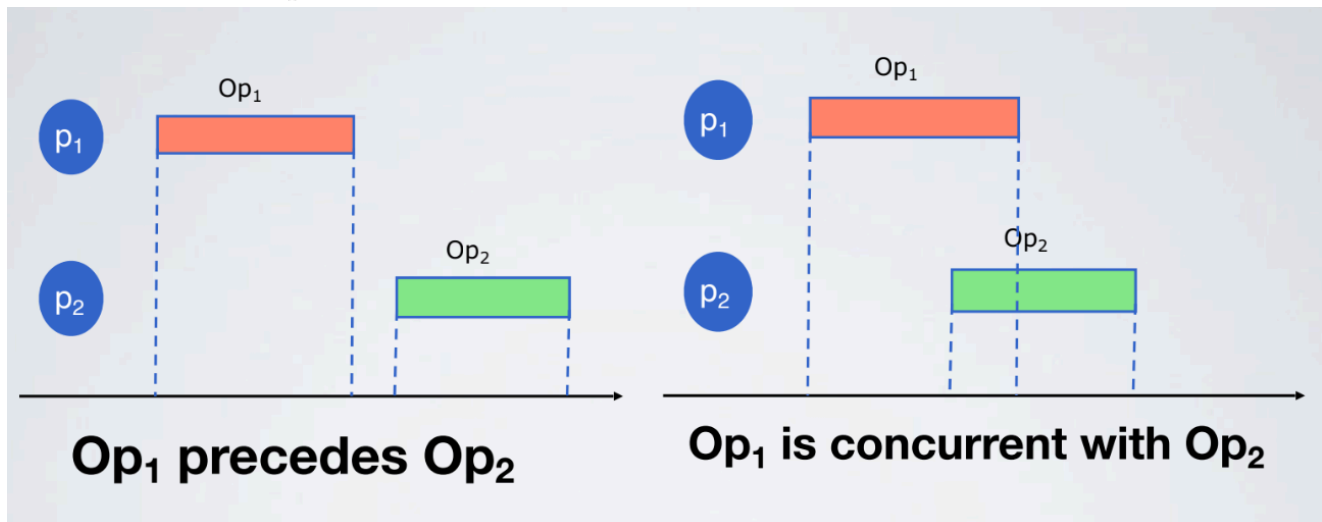


An operation fails when someone invoke the operation, and then dies before operation terminates on the process that invoked it.

How do we define operation precedence?

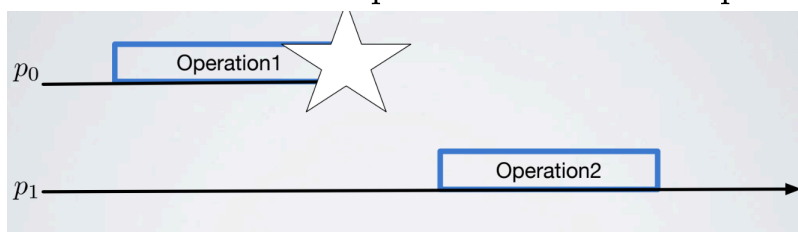
- The execution of an operation invoked by a process p , is the time interval defined by the invocation event and the return event.
- Given two operations o and o' , **o precedes o'** if the return event of o precedes the invocation event of o' .
- An operation o invoked by a process p may precede an operation o' invoked by p' only if o completes.
- If no precedence relation between two operations can be defined, they are said to be **concurrent**.

Let's see an example:



So an operation o may precede operation o' coming from another process if o has already seen a return.

What is the relationship between these two operations?



They are concurrent, as o_1 never returns (so we can extend it forever), and thus o_2 is concurrent with o_1 .

Let's now see the registers semantic. It is a set of rules that specify how a register behaves.

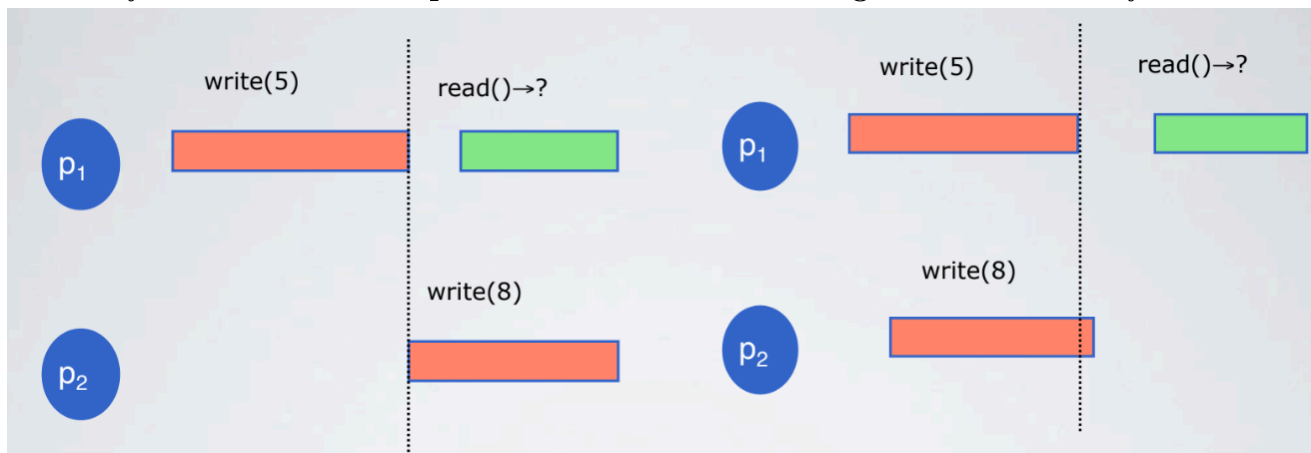
The easier semantic for us is the strictly serialized semantic. With it, a process can

write on a register only by taking a lock. With it, we are able to understand exactly the value of a read, as operations can not be concurrent.

There are two specifications:

- **Liveness.** Each operation eventually terminates
- **Safety.** Each read operation returns the last value written

Let's say that I now allow processes to read on the register concurrently.



What value do I return in the read operation? It's not clear.

A similar example to this is given with the presence of failures.



Our goal will be to define a set of useful semantics.

In the literature there are a lot of semantics, but we'll take a look at:

- regular consistency
- sequential consistency
- linearizability/atomic consistency

Let's start by taking a look at regular consistency. We'll see the implementation of Regular Registers in Fail Stop and Fail Silent.

Usually a register is associated with a pair of value (X, Y) , where X processes can write and Y processes can read.

- $(1, 1)$ denotes a register where only a process can write and only a process can read.

$(1, 1)$ does not mean that a system is made just by two processes, there could be multiple processes.

The most common register is $(1, N)$, with a single writer and everyone in the system can read. A process could be both writer and reader.

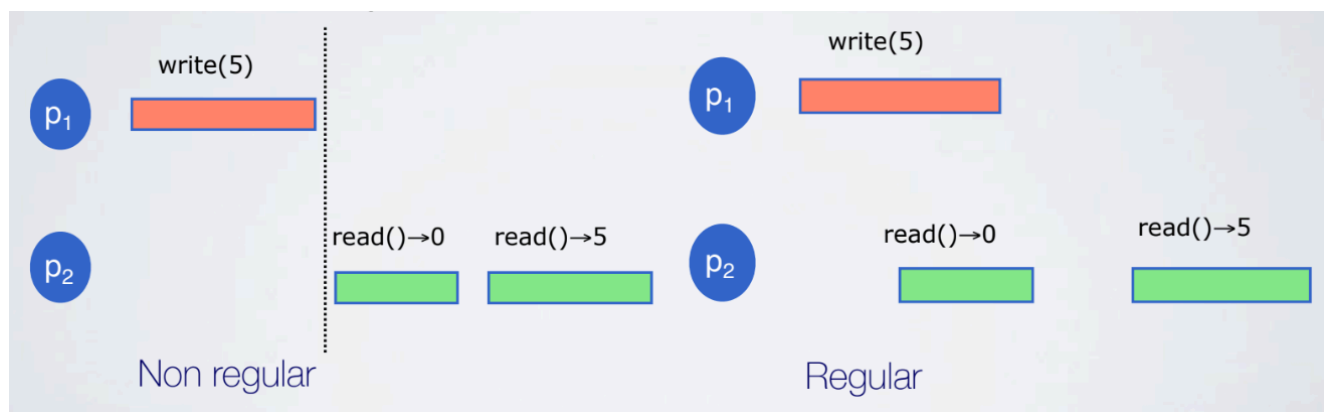
When the algorithm starts, we know the identity of the writers.

We also have (N, N) as a register, with everyone having the permission to write and read on the register.

We'll see the $(1, N)$ regular register at first.

The regular register has two properties:

- Termination: if a correct process invokes an operation, then the operation eventually receives the corresponding confirmation. *if we are correct, we don't care about the others*
- Validity: A read operation returns the last value written or the value concurrently written. *So if there is a concurrent operation happening, we either read the last value OR the one we are writing.*

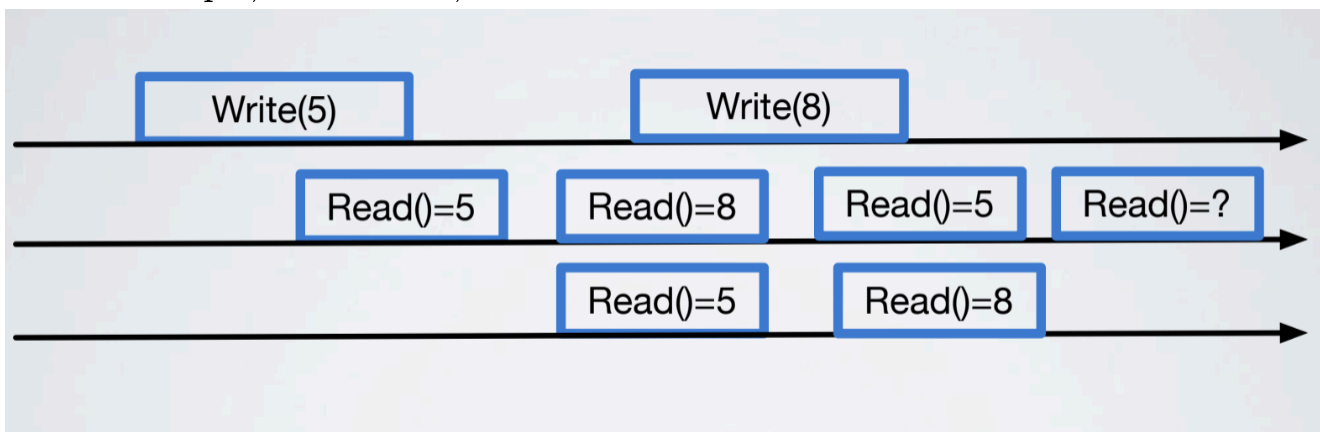


NOTE: In a regular register, a process can read a value v and then a value v' , even if the writer has written v' and then v , as long as the write and the read operations are concurrent

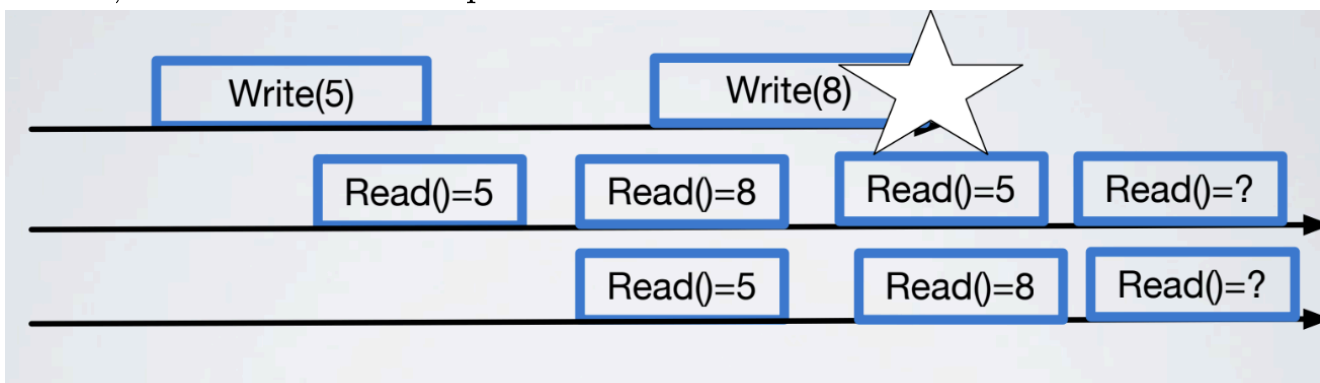


The last read is still correct, as we could read either the value someone is concurrently writing, or the value before. If I read after the last, I would read 6.

In this example, we return 8, as it's the last value written.



In this other case we could return 8 or 5, as we could extend the write operation forever, so it's a concurrent operation.



So we could have executions with 5, 5, 5, 8, 8, 5 or 8, 8 as the values of read.

Now we implement the primitive. We assume P . The Regular Register will have the following events:

- Read: invokes a read operation on the register
- Write| v : invokes a write operation with value v on the register.
- ReadReturn| v : completes a read operation on the register with return value v
- WriteReturn: completes a write operation on the register.

We also have two properties:

ONRR1: Termination: If a correct process invokes an operation, then the operation eventually completes.

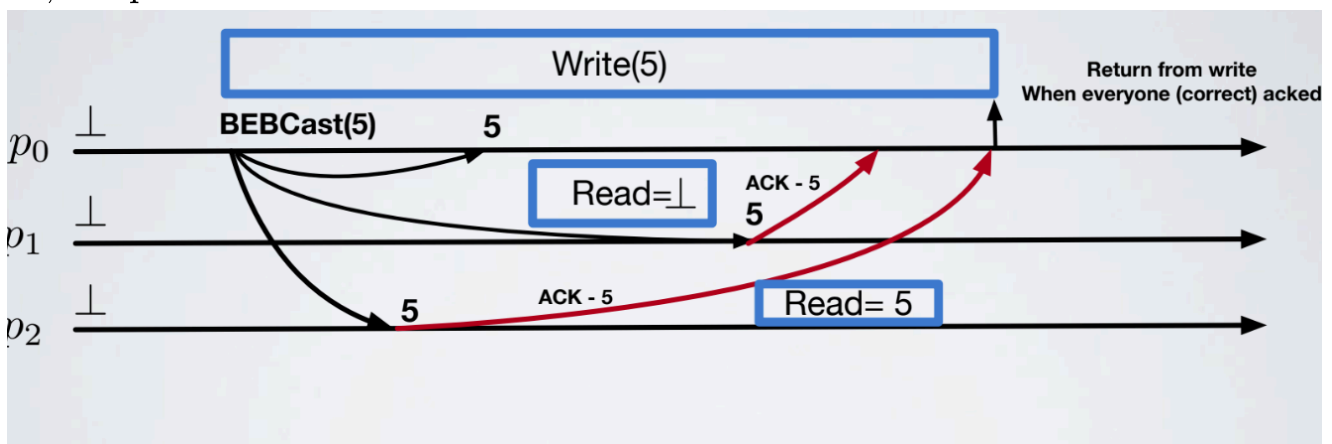
ONRR2: Validity: A read that is not concurrent with a write returns the last value written; a read that is concurrent with a write returns the last value written or the value concurrently written.

So, how do we implement it? We'll need a buffer to store the shared data. So each process has it's local value, and then we'll have a mechanism that each value is consistent.

The main idea would have the writer do all the job, so he has to ensure the consistencies of copies.

Now, if p_0 is a writer, he could do a write operation, but when do we return? If we return before we are sure that everyone has received the message, we could have a problem with consistency. So, we could implement an ACK solution. So, we terminate when $\text{correct} \subseteq \text{ack}[m]$.

So, the pattern will be:



We are in fail stop, so we assume P , p2p link, and BEB. What is the idea of the algorithm?

- Each process stores a local copy of the register
- Read-One: each read operation returns the value stored in the local copy of the register
- Write-All: each write operation updates the value locally stored at each process the writer consider to have not crashed
- A write completes when the writer receives an ack from each process that has not crashed

Let's see the code:

```

upon event  $\langle onrr, Init \rangle$  do
     $val := \perp$ ;
     $correct := \Pi$ ;
     $writeset := \emptyset$ ;

upon event  $\langle \mathcal{P}, Crash \mid p \rangle$  do
     $correct := correct \setminus \{p\}$ ;

upon event  $\langle onrr, Read \rangle$  do
    trigger  $\langle onrr, ReadReturn \mid val \rangle$ ;

upon event  $\langle onrr, Write \mid v \rangle$  do
    trigger  $\langle beb, Broadcast \mid [WRITE, v] \rangle$ ;

upon event  $\langle beb, Deliver \mid q, [WRITE, v] \rangle$  do
     $val := v$ ;
    trigger  $\langle pl, Send \mid q, ACK \rangle$ ;

upon event  $\langle pl, Deliver \mid p, ACK \rangle$  do
     $writeset := writeset \cup \{p\}$ ;

upon  $correct \subseteq writeset$  do
     $writeset := \emptyset$ ;
    trigger  $\langle onrr, WriteReturn \rangle$ ;

```

We start and everyone sets val to $\perp$. In the $writeset$, we'll store the ack.

Each process will execute the same code. However, as we are in $(1, N)$, only one process can execute the *Write* handler.

Let's say that the writer p_1 decides to write value 5, then we broadcast it to everyone in the system. When a process p_2 receives value 5, he triggers a *beb Deliver*, setting val to 5 and send back an ACK to p_1 . p_1 then receives the ACK, and updates the $writeset$ ack including the id of the process sending the ACK. Once $correct \subseteq writeset$ we set $writeset = \emptyset$ and return.

So, for a single write operation, how many messages do we need? $2n$.

How many messages do we generate for a read? 0, as it's local.

The message delay is given by 2, as we have one unit for the BEB, and one other for the ACK. A read operation has 0 message delay.

Let's take a look at the correctness of the algorithm:

- Termination

The read is trivial, as it's local.

The write, from the properties of the communication primitives and from the completeness property of P .

- Validity

Because of the strong accuracy property of the perfect failure detector, each write operation can complete only after all processes that have not crashed updated their local copy of the register. *In order to terminate, all the correct are updated to our value.* Now we may have two cases:

- The read operation is not concurrent with the last write that has been invoked, and then the process will read the last value written.
- The read operation is concurrent with the last write. For the no creation property of the channels, the value returned is either the last value written or the one being written. This latter is concurrent with the read operation.

As we are using a BEB, we could have the writer crash and then, as write and read operations are concurrent, different processes could have different values when reading.

Now, we have the RR built on BEB and P . We could weaken P with $\Diamond P$. Is the algorithm correct if we use $\Diamond P$? No, we could have that a process is not correctly flagged as dead, and so we could return without the other process actually updating val, and so the other process could read a value without concurrent operations, reading a wrong value.

Is it possible, now, to adapt the algorithm for Fail Silent? Can we use Quorums? Yes, but we also add an assumption to the system, by which we have that the majority of processes are correct ($\frac{n}{2} + 1$). When does the writer terminate its write? When he receives the majority of ACKs.

Can a read operation be local now? No, if a correct process reads a value even though he has not received the new value and the writer already terminated the write, it could read a value as of now. So, now we ask to everyone for their value, and when we receive the majority of answers, we'll know what the updated value is.

Now, how do we actually know what is the value? We can't use the majority of the

values (see examples in slide for clearer understanding).

What is the easiest mechanism? The timestamps. The writer will write something like value 1 and timestamp $0 < 1, 0 >$, and $< 2, 1 >$ (for example) and so on.

We'll have the Majority Voting Regular Register.

Here is the idea:

- Each process locally stores a copy of the current value of the register
- Each written value is univocally associated to a **timestamp**
- The writer and the reader processes use a set of witness processes, to track the last value written
- Quorum: the intersection of any two sets of witness processes is not empty
- “Majority Voting”: each set is constituted by a majority of processes
- The reader takes the value with the greatest timestamps among the processes in quorum.

Now, we see the algorithm:

```
upon event  $\langle onrr, Init \rangle$  do  
   $(ts, val) := (0, \perp)$ ;  
   $wts := 0$ ;  
   $acks := 0$ ;  
   $rid := 0$ ;  
   $readlist := [\perp]^N$ ;  
  
upon event  $\langle onrr, Write \mid v \rangle$  do  
   $wts := wts + 1$ ;  
   $acks := 0$ ;  
  trigger  $\langle beb, Broadcast \mid [WRITE, wts, v] \rangle$ ;  
  
upon event  $\langle beb, Deliver \mid p, [WRITE, ts', v'] \rangle$  do  
  if  $ts' > ts$  then  
     $(ts, val) := (ts', v')$ ;  
  trigger  $\langle pl, Send \mid p, [ACK, ts'] \rangle$ ;  
  
upon event  $\langle pl, Deliver \mid q, [ACK, ts'] \rangle$  such that  $ts' = wts$  do  
   $acks := acks + 1$ ;  
  if  $acks > N/2$  then  
     $acks := 0$ ;  
    trigger  $\langle onrr, WriteReturn \rangle$ ;  
  
upon event  $\langle onrr, Read \rangle$  do  
   $rid := rid + 1$ ;  
   $readlist := [\perp]^N$ ;  
  trigger  $\langle beb, Broadcast \mid [READ, rid] \rangle$ ;  
  
upon event  $\langle beb, Deliver \mid p, [READ, r] \rangle$  do  
  trigger  $\langle pl, Send \mid p, [VALUE, r, ts, val] \rangle$ ;  
  
upon event  $\langle pl, Deliver \mid q, [VALUE, r, ts', v'] \rangle$  such that  $r = rid$  do  
   $readlist[q] := (ts', v')$ ;  
  if  $\#(readlist) > N/2$  then  
     $v := highestval(readlist)$ ;  
     $readlist := [\perp]^N$ ;  
  trigger  $\langle onrr, ReadReturn \mid v \rangle$ ;
```

With wts we copy the sequence number of the writer. acks is a counter increased when we receive an ACK. rid is an id we put inside the read request. readlist is where I put the value received from other processes when I'm reading.

When someone writes we increase wts, set acks to zero, and then I send to

everyone the wts and v . When someone receives a message with a certain ts' and v' it modifies the local value only if $ts' > ts$, and in that case we update (ts, val) . I then send an ACK.

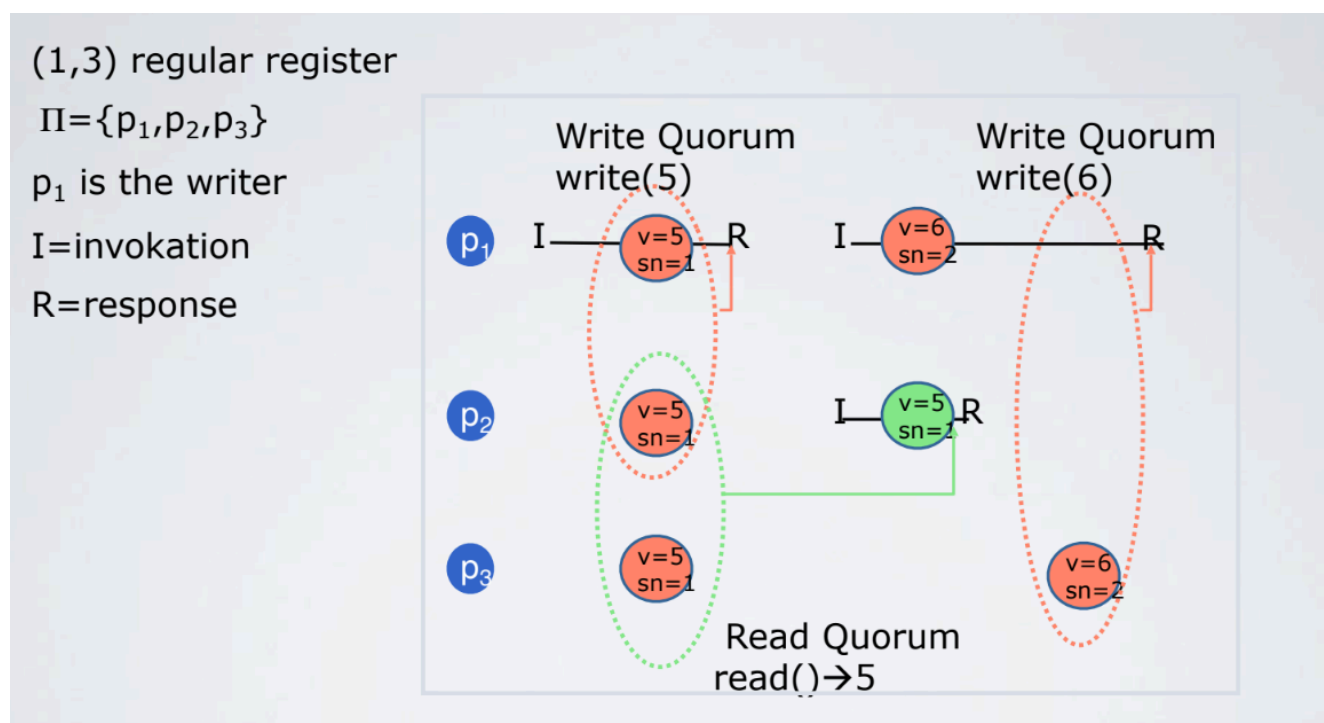
When the writer delivers an ACK, we check the timestamp is equal to wts . When the counter of ACKs for the request is greater than $\frac{N}{2}$, we set the acks to zero and trigger the return.

When someone wants to read, we'll increase the rid ...

missing part

If in readlist there are $\frac{N}{2} + 1$ elements different from bottom, then we go inside the if. We'll take the element with the maximum timestamp, update the readlist and trigger the read return.

Let's now see a few executions:



Let's say that at first everyone receives the message, and then because there is a quorum the writer will return. Then p_2 will ask to read the value, and only p_2 and p_3 will answer, and it's not a problem as we have a quorum.

So, what is the message complexity of a write? It's $2N$, as we send the message to all process, and all answers. For the read? Always $2n$.

The message delay is 2 for the write, and this is true also for the read.

This complexity is higher from the one we've seen before, but it's a trade off considering we are in a weaker system.

What is the resiliency (number of failures we tolerate)? It's $f \leq \frac{n}{2} - 1$.

In the previous algorithm, we could even tolerate $f = n - 1$ failures.

If the number of failures here is $\geq \frac{n}{2}$ we could violate termination, as an operation could never end as it would be waiting forever for an ACK.

If we violate validity over the threshold of failures, then even under the threshold the algorithm is not correct.

It remains now to be seen sequential consistency and atomic consistency.

We'll see three kind of consistency property:

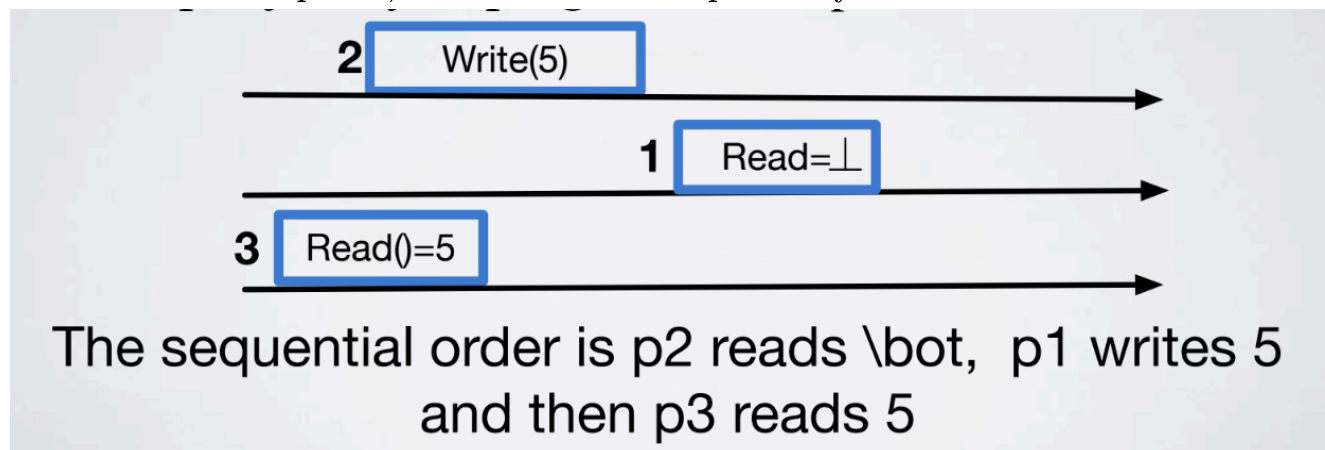
- regular
- sequential
- atomicity

The sequential consistency was invented by Lamport, and the definition is:

"The result of any execution is the same as if the (read and write) operations by all processes on the data store were executed in some sequential order and the operations of each individual process appear in this sequence in the order specified by its program."

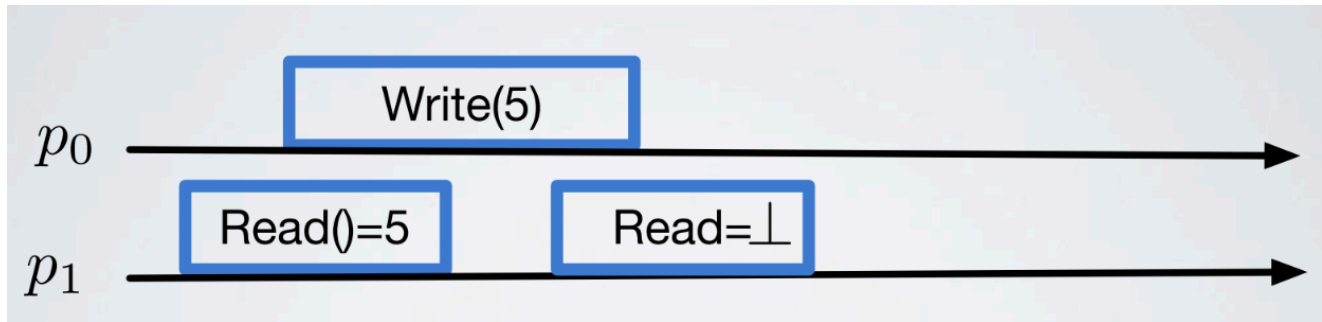
An execution is sequential consistent if there is some fictional sequential order that respect the order of operations seen locally by the process.

In the case of the photo, the execution is sequentially consistent.



The only important thing is that we don't swap operations on the same process. So, another definition is: There exists a global ordering, that respects the local ordering seen by each process.

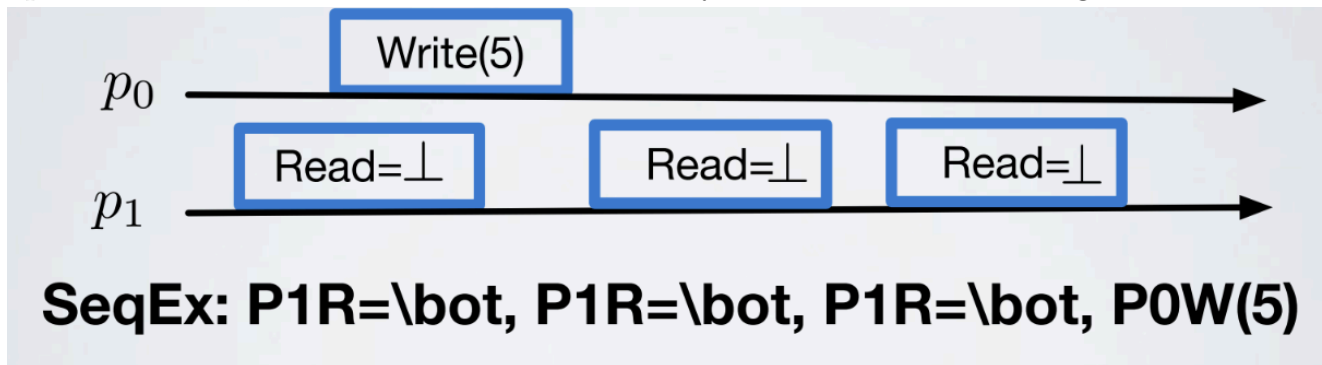
Let's see another example:



Is this regular? Yes.

How do we fix elements for sequential consistency? There is no way, we'll have to swap the local order of a process (for process p_1 , we cannot do that in sequential consistency, so we can't swap the local order seen by a process). *We cannot swap what a single process do.*

This other execution is sequential consistent, as we could have all the read operations first, and then the write. It's okay as we are not inverting the order.



As for the fact there are some executions that are regular but not sequential consistent, and viceversa, then they are orthogonal.

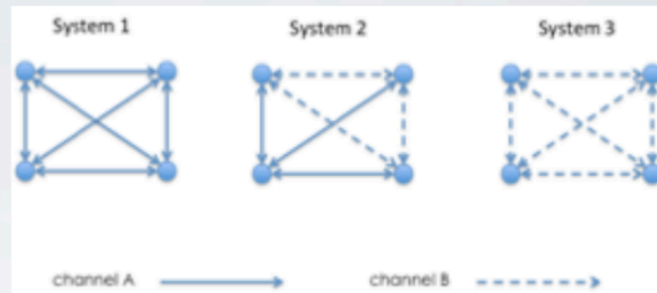
Exercises

Ex 4: Let channel A and channel B be two different types of point-to-point channels satisfying the following properties:

- channel A: if a correct process p_i sends a message m to a correct process p_j at time t , then m is delivered by p_j by time $t+x$.

- channel B: if a correct process p_i sends a message m to a correct process p_j at time t , then m is delivered by p_j with probability p_{cons} ($p_{\text{cons}} < 1$).

Let us consider the following systems composed by 4 processes p_1, p_2, p_3 and p_4 connected through channels A and channels B.



Assuming that each process p_i is aware of the type of channel connecting it to any other process, answer to the following questions:

1. is it possible to design an algorithm implementing a leader election primitive in system 1 if any process can fail by crash?
2. is it possible to design an algorithm implementing a leader election primitive in system 2 if only processes having an outgoing channel of type B can fail by crash?
3. is it possible to design an algorithm implementing a leader election primitive in system 2 if any process can fail by crash?
4. is it possible to design an algorithm implementing a leader election primitive in system 3?

For each point, if an algorithm exists write its pseudo-code, otherwise show the impossibility.

1. .

Yes, because this is a synchronous system.

For the pseudocode we simply say that we use the le .

2. .

Yes, as we can see the system as synchronous (using the synchronous channels to communicate, having 2Δ as a delay).

Here we say that we use the bottom left process as middleman for pseudocode.

3. .

No, because if the only process using A channels we remain with a system that is asynchronous, and we cannot elect leader anymore, as we cannot detect a failure anymore.

4. .

No, for the same reason as above.

Now, let's see another exercise.

Consider a distributed system composed by n processes $\{p_1, p_2, \dots, p_n\}$ that communicate by exchanging messages on top of a line topology, where p_1 and p_n are respectively the first and the last process of the network.

Initially, each process knows only its left neighbour and its right neighbour (if they exist) and stores the respective identifiers in two local variables LEFT and RIGHT.

Processes may fail by crashing, but they are equipped with a perfect oracle that notifies at each process the new neighbour (when one of the two fails) through the following primitives:

- **Left_neighbour(p_x):** process p_x is the new left neighbour of p_i
- **Right_neighbour(p_x):** process p_x is the new right neighbour of p_i

Both the events may return a NULL value in case p_i becomes the first or the last process of the line.

Each process can communicate only with its neighbours.

A line topology is something like that $\boxed{p_1} - \boxed{p_2} - \boxed{p_3} - \boxed{p_4}$.

When a neighbor process fails, we reconnect the line with the nearest process in that direction.

So we have a Neighbor oracle and P2P. We cannot use P or le . We have to implement starting from this the RB.

A difficult property is the agreement, by which if a correct process receives a message, then every other correct process will receive it.

To solve this exercises, we first start with the implementation without failures,, and then we implement it.

If p_5 wants to send a message, he simply sends it left. When p_4 receives it, it could simply deliver it. And then it starts a chain of sending left and delivering, ecc.

If p_3 was the sender, we send to both directions.

What if we add failures? When we have a new neighbor (when one crashed), we send the message again.

In the code, we dont care about how the oracle works, it will trigger itself the event new left/right neighbor.